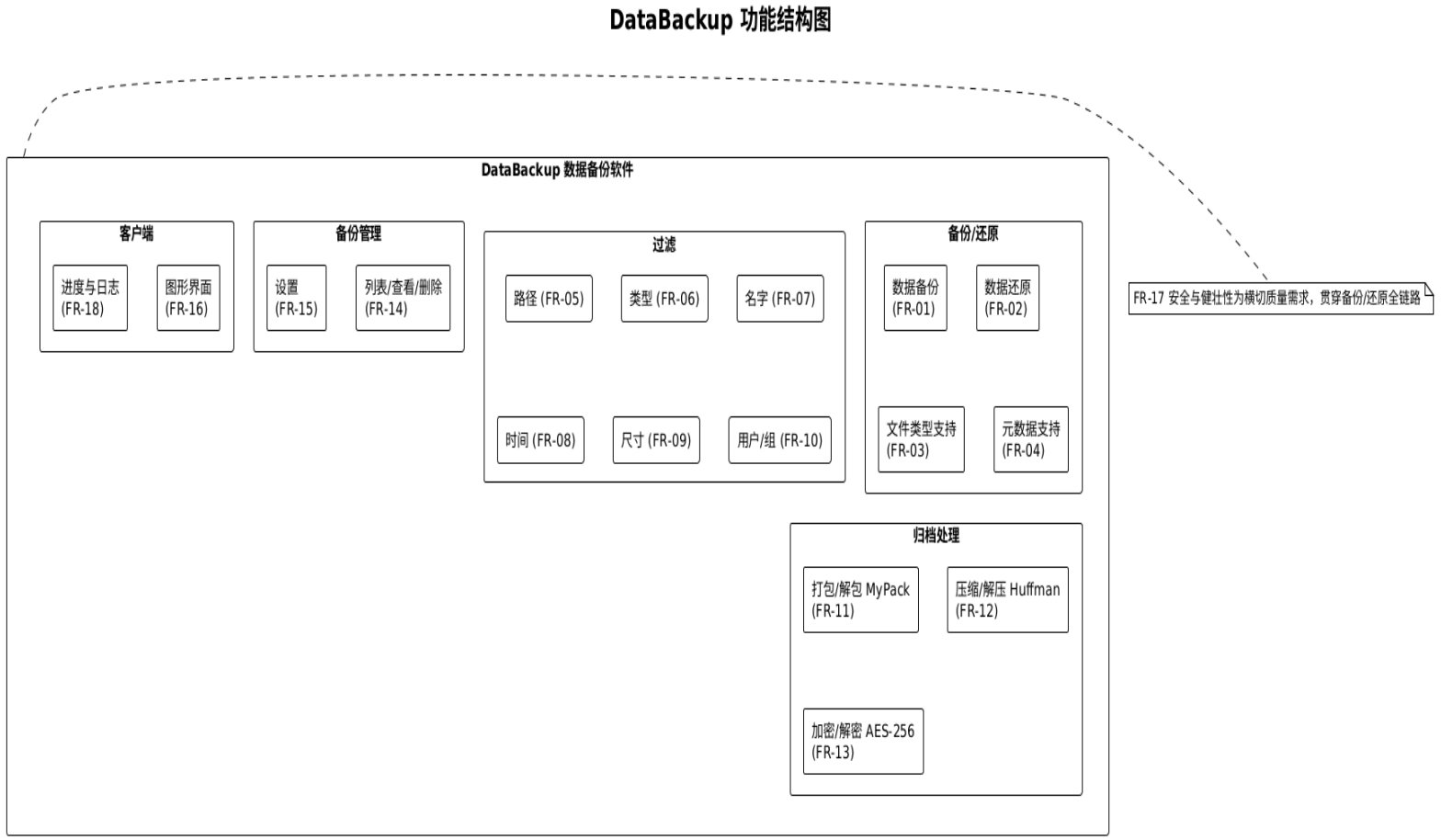
DataBackup

Linux 目录树备份与还原软件

组长：王科瑾（单人组 · AI 辅助开发）
电子科技大学《软件开发综合实验》

项目定位与功能总览

一句话：把任意 Linux 目录树备份为单个备份文件，并原样还原（数据 + 全部文件类型 + 元数据）



基础 40 分

数据备份 / 数据还原

扩展项全做

7 种文件类型 · 元数据

六维过滤 · MyPack

Huffman · AES-256

Electron 图形界面

需求与用例：全程编号可追踪

18 / 18

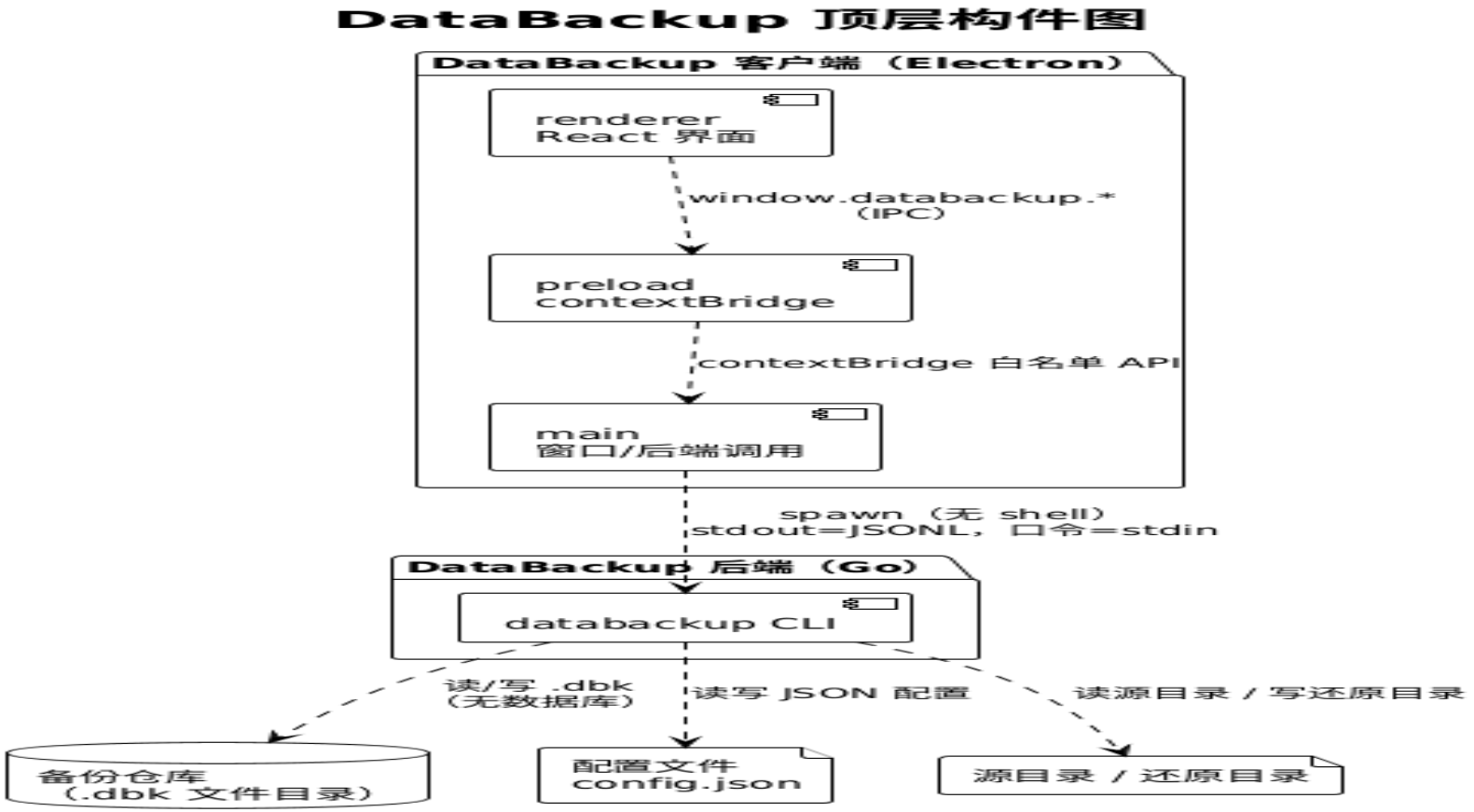
FR-01 ~ FR-18 全部实现

UC-01 ~ 07 覆盖备份、还原、管理、设置、取消、口令、进度七个用例；每个 FR 至少 1 个自动化用例（FR ↔ TC 追踪表）

FR-01/02	备份与还原（基础）
FR-03/04	7 种文件类型 + 纳秒元数据
FR-05~10	路径 / 类型 / 名字 / 时间 / 尺寸 / 用户组过滤
FR-11~13	MyPack / Huffman / AES-256 （全自研）
FR-14~15	仓库管理与设置
FR-16~18	图形界面 / 安全健壮 / 进度日志

总体架构

Go 编译型后端 + Electron 客户端，JSON Lines 协议为唯一接口（冻结契约 + 双端类型一致性测试）



1 零依赖

后端生产代码零第三方库，扩展功能全部自研

2 契约冻结

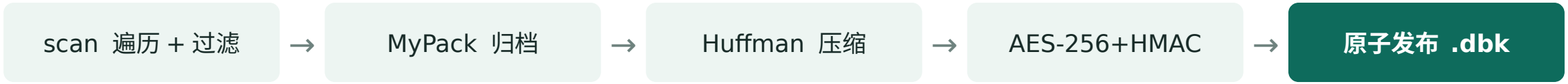
CLI/JSONL 协议文档化，Go/TS 类型测试比对

3 进程隔离

客户端 spawn 子进程，逻辑全在后端

数据流水线

scan （遍历 + 过滤）→ pack （ MyPack → Huffman → AES+HMAC ）→ .partial → 硬链接原子发布



先校验后落盘

还原先整文件校验（ MAC/SHA ），通过前不写任何文件

原子发布

硬链接发布为 .dbk ，遇同名 E_EXISTS ； 失败 / 取消删 .partial 无残留

进度节流

JSON Lines 实时输出，每阶段 ⩽ 10 次 / 秒且阶段结束必发

完整顺序图（备份 / 还原加密 /dry-run/ 列表删除）见系统设计文档 §4.1

关键技术一：自研 MyPack 归档格式



小端二进制 · CRC-32 校验 · 纳秒时间 · 全部 7 种文件类型

- 安全： Reader 拒绝绝对路径、 .. 段、已出现软链接之下的路径、越界硬链接目标
- 长文件名溢出到扩展块，头记录定长 512B （仿 tar）
- 黄金文件往返逐字节一致；篡改任何字段检出

488 万

FuzzReader 30s 执行次数，零 panic

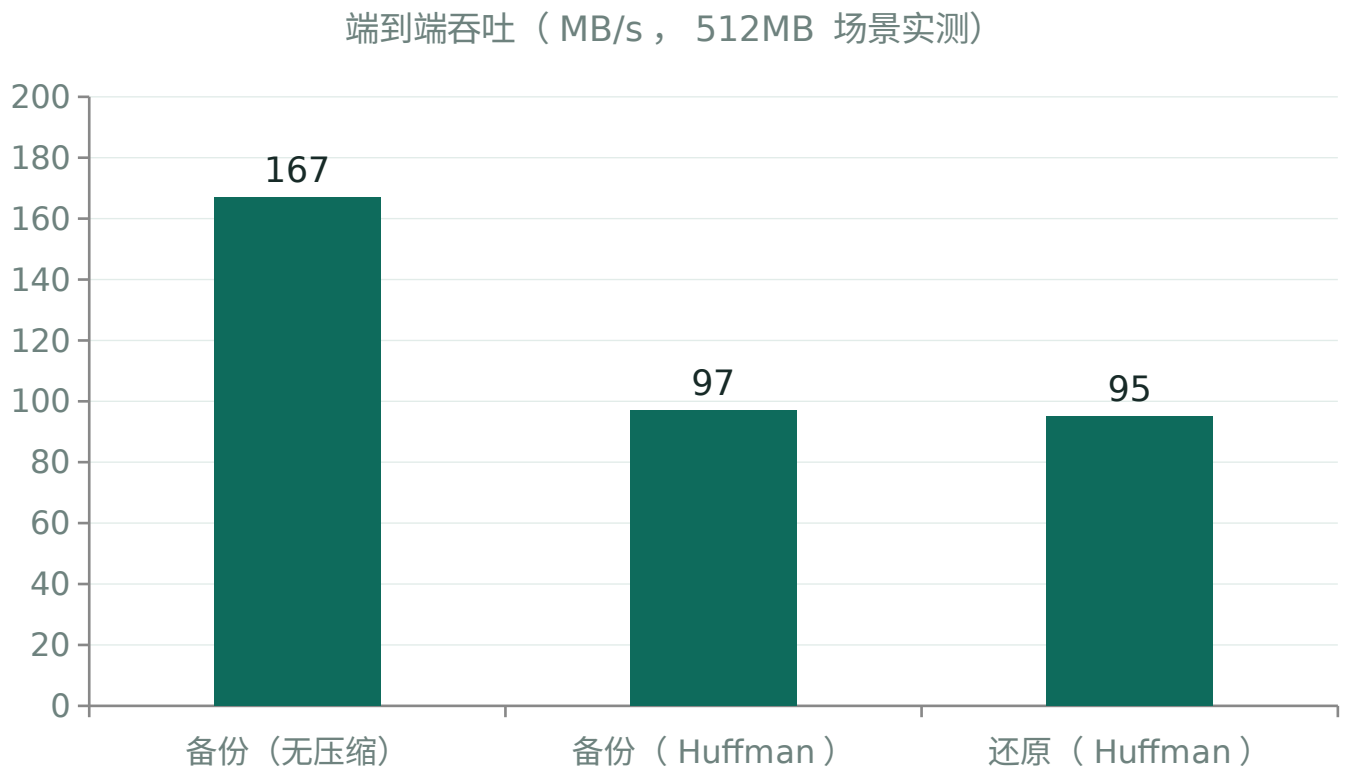
关键技术二：自研 Huffman 压缩

1MiB 分块流式规范编码，码长表入块头，内存恒定

0.616

512MB 文本实测压缩率（330.8MB）

- 规范码表可自校验，解码对非法输入返回错误
- 随机数据不膨胀（0.999+），跨块流式处理
- 10,000 个小文件备份 0.16s、还原 0.36s



关键技术三：手写加密栈

口令 → PBKDF2 （10 万轮， 214ms ）→ AES-256-CBC + PKCS#7 → encrypt-then-MAC



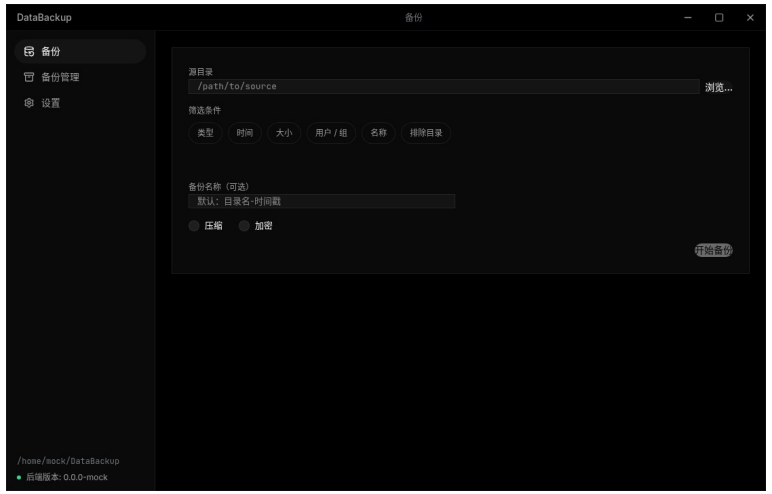
- 全部手写，仅测试中允许标准库对照组（课程自研要求）
- 公开向量锚点： FIPS-197 、 NIST SHA-256 、 RFC 2104/7914 全过
- 容器先整文件校验再返回明文流：篡改任一字节 / 截断 / 错误口令必被检出
- 口令只经 stdin 传递，不进参数表 / 环境变量 / 日志

E_AUTH

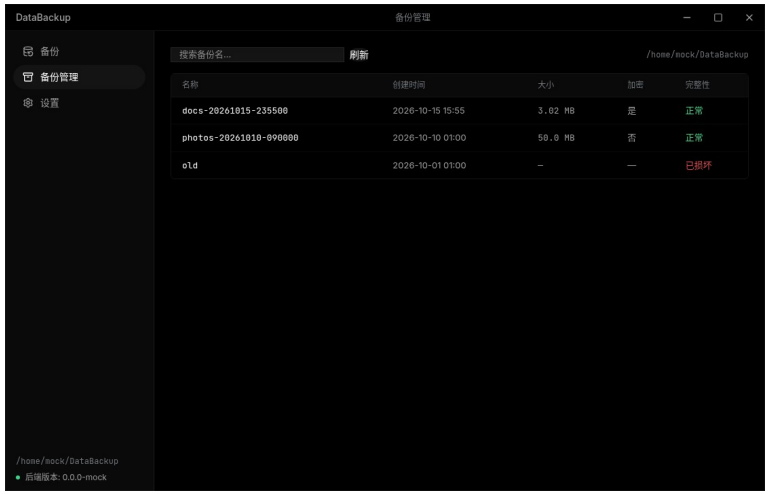
错误口令退出码 3 ，客户端 3 次重试后回退

Electron 图形客户端

三页面真实截图（Playwright 界面回归基线即所见即所得）



备份：编排 + 六筛选



管理：列表 / 还原 / 删除



设置：仓库路径

React 19 + zustand 状态机（9 态全转移有单测）；进度实时（JSON Lines）；口令经 stdin 注入

端到端：界面完成「过滤 + 压缩 + 加密」备份 → 口令还原 → 目录树与源一致（真实后端二进制）

工程化：并行开发与质量门禁

40+

任务按波次 / 泳道并行

按模型强度分级调度（强 / 中 / 轻），
两日冲刺完成全部功能冻结

AGENTS.md 规则

自研约束 / 零依赖 / 无学号，任何 agent 进仓即受约束

分级调度

roadmap 40+ 任务，依赖分波次、强度分级

review 循环

并行深审 subagent + 门禁复跑 + P0~P3 分级

自动化门禁

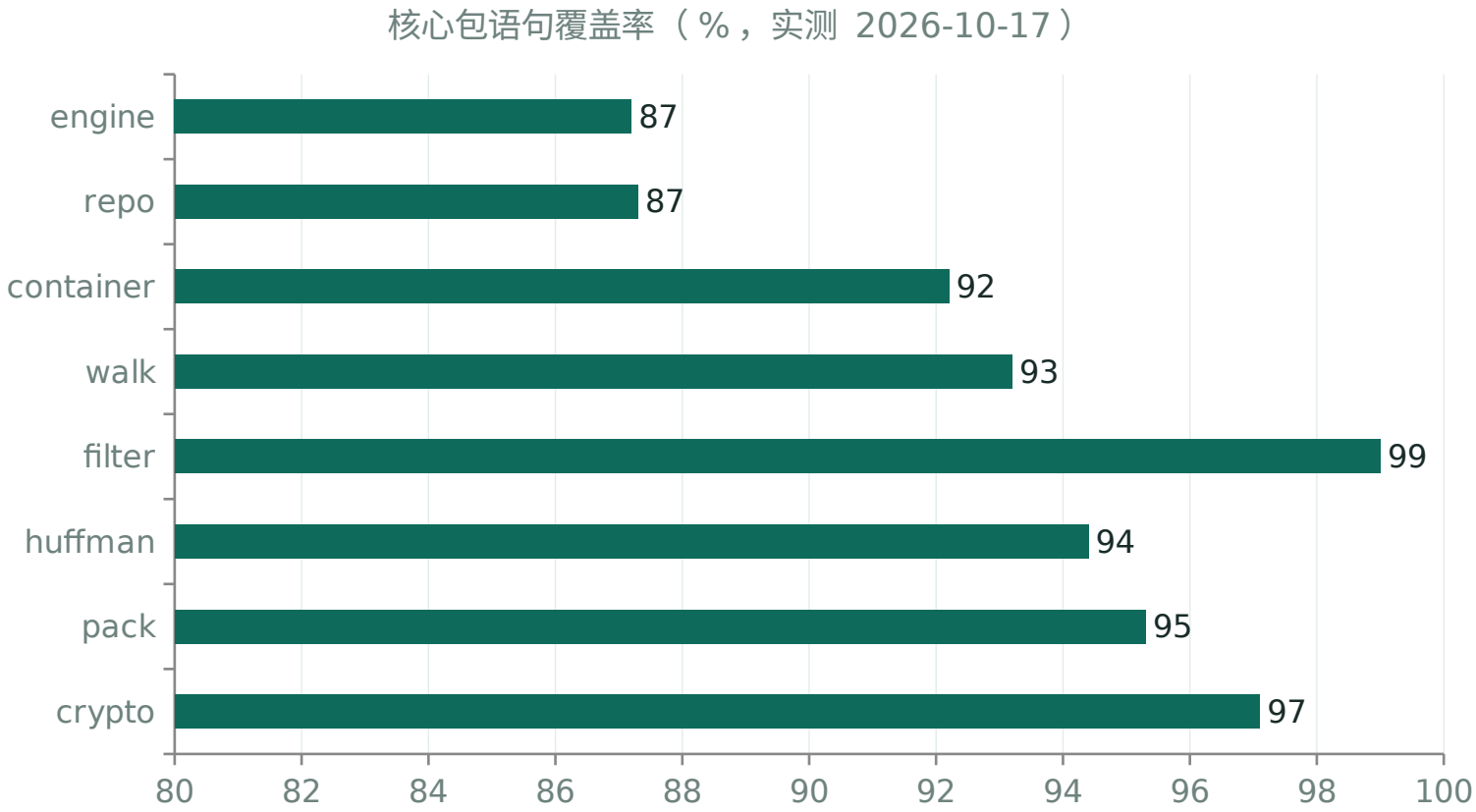
make check / check-all + CI 四 job 全绿才可合并

测试证据（一）：规模与覆盖

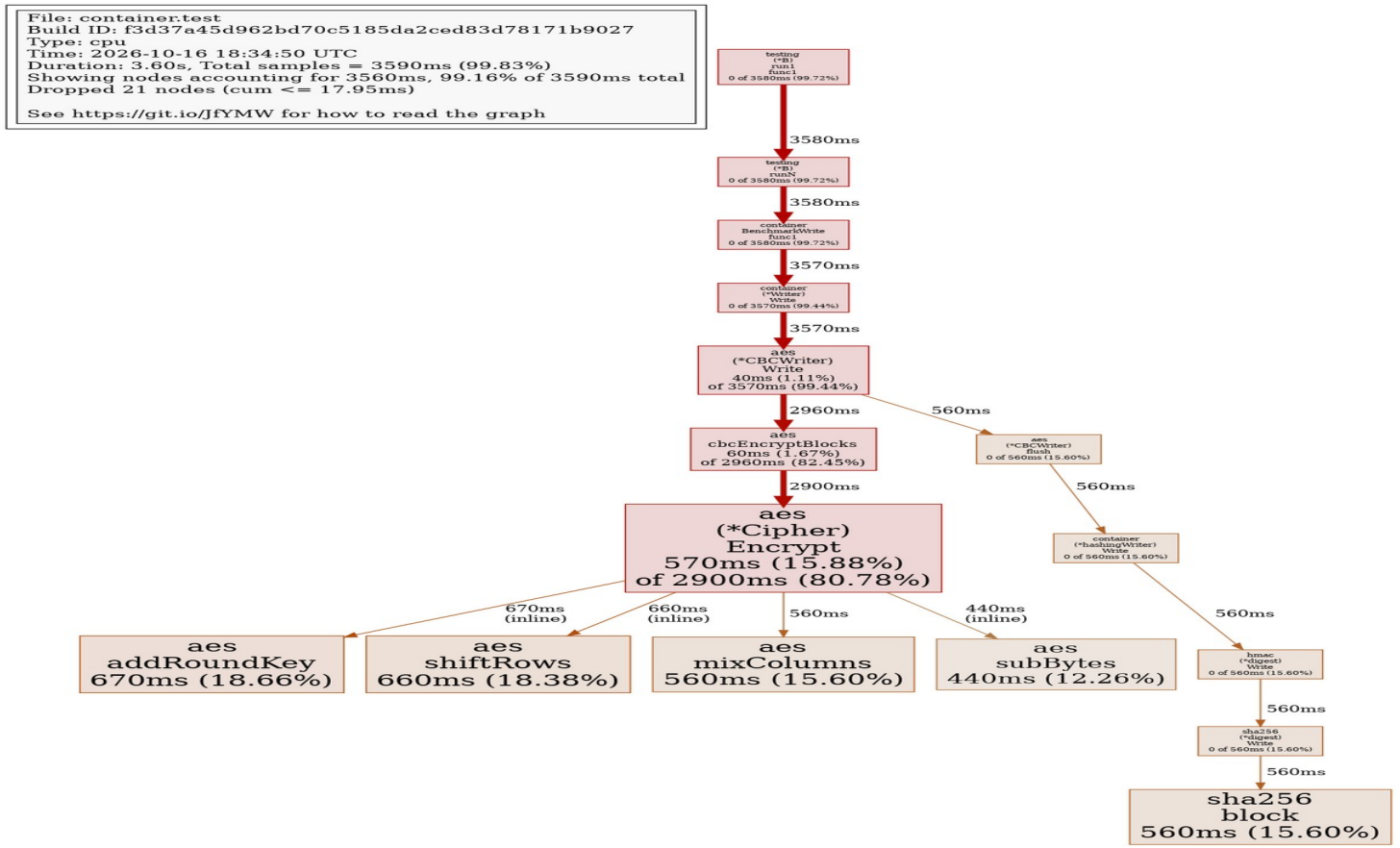
400+

用例编号内嵌代码，可直接检索

- 32 组合过滤 × 压缩 × 加密矩阵全过
- race 全绿 · 静态检查零告警
- 核心包覆盖率全部 ≥ 80%



测试证据（二）：健壮性与性能剖析



pprof 火焰图：热点集中于 AES 轮函数与 CBC 链式异或（符合手写实现预期）

10,000

小文件备份 0.16s

0.16s / 0.36s

万文件备份 / 还原

SIGTERM

取消退出码 4，无 .partial 残留

488 万次

fuzz 执行零 panic

交付物与运行

make package 一键产出； WSL2 + WSLg 运行说明（实测通过）

DataBackup-0.1.0.AppImage

139.8MB · 免安装单文件

libfuse2t64 或 --appimage-extract-and-run

databackup-client_0.1.0_amd64.deb

107.6MB · 标准安装

dpkg -i + apt-get -f，内置后端二进制

databackup CLI

3.7MB · 独立可执行

脚本化备份 / 还原， JSON Lines 输出

全部产物内置后端于 resources/bin/databackup； GUI 与 CLI 共用同一二进制，版本一致性由构建保证

≤3 分钟演示视频：界面完成过滤 + 压缩 + 加密备份 → 错误口令 → 还原成功 → diff 验证

AI 辅助开发实践

规则约束 + 分级调度 + 独立审查 + 自动化门禁，四道闸门缺一不可

1 → **2** → **3** → **4**

规则入口	任务拆分	实现与审查	门禁合入
AGENTS.md：自研约束 / 零第三方依赖 / 不提交学号	roadmap 波次泳道 + 模型强度分级（强 · 中 · 轻）	多 agent 并行实现；review-agent 循环深审	先写测试后实现；check-all + CI 全绿才合并

两日冲刺（10-16 ~ 10-17）完成 W3 集成 → C3 联调 → C4 打包 → 测试证据，功能冻结较计划提前一天

AI 产出经独立审查发现的典型深层问题：e2e 截图基线随打包流内环境元数据漂移——根因取证与修复全程留档

总结

- 基础 + 全部扩展项实现， FR ↔ TC 全程可追踪
- 自研格式 / 压缩 / 加密：公开向量 + 488 万次 fuzz 背书
- 从需求到发布的完整工程化闭环（测试 / 打包 / 文档 / CI ）

谢谢！

演示见视频 / 现场 · 组长：王科瑾